

This approach still has its problems. Specifically, it works well when every function in the call hierarchy has only one parent, but it breaks down when we try to profile a function that is called by more than one parent function. The reason for this should be pretty obvious. We're statically declaring our sample bins *as if* every function can only appear once in the function call hierarchy, but actually the same function can reappear many times in the tree, each time with a different parent. The result can be misleading data, because a function's time will be included in one of the parent bins, but really should be distributed across all of its parents' bins. Most game engines don't make an attempt to remedy this problem, since they are primarily interested in profiling coarse-grained functions that are only called from one specific location in the function call hierarchy. But this limitation is something to be aware of when profiling your code with a simple in-engine profile of the sort found in most game engines.

Of course, it is also possible to write a much more sophisticated profiling system that handles nested instances of AutoProfile properly. This is an example of the many trade-offs one makes when designing a game engine. Do we invest the engineering time to create a fully hierarchical profiler? Or, do we make do with something simpler and invest those programming resources elsewhere? Ultimately, it's up to you.

We would also like to account for how many *times* a given function is called. In the example above, we know that each of the functions we profiled is called exactly once per frame. But other functions, deeper in the function call hierarchy, may be called more than once per frame. If we measure function `x()` to take 2 ms to execute, it's important to know whether it takes 2 ms to execute on its own, or whether its execution time is 2 ms but it was called 1,000 times during the frame. Keeping track of the number of times a function is called per frame is quite simple—the profiling system can simply increment a counter each time a sample is received and reset the counters at the start of each frame.

10.8.2 Exporting to Excel

Some game engines permit the data captured by the in-game profiler to be dumped to a text file for subsequent analysis. I find that a comma-separated values (CSV) format is best, because such files can be loaded easily into a Microsoft Excel spreadsheet, where the data can be manipulated and analyzed in myriad ways. I wrote such an exporter for the *Medal of Honor: Pacific Assault* engine. The columns corresponded to the various annotated blocks, and each row represented the profiling sample taken during one frame of the game's

execution. The first column contained frame numbers and the second actual game time measured in seconds. This allowed the team to graph how the performance statistics varied over time and to determine how long each frame actually took to execute. By adding some simple formulae to the exported spreadsheet, we could calculate frame rates, execution time percentages and so on.

10.9 In-Game Memory Stats and Leak Detection

In addition to runtime performance (i.e., frame rate), most game engines are also constrained by the amount of memory available on the target hardware. PC games are least affected by such constraints, because modern PCs have sophisticated virtual memory managers. But even PC games are constrained by the memory limitations of their so-called “min spec” machine—the least-powerful machine on which the game is guaranteed to run, as promised by the publisher and stated on the game’s packaging.

For this reason, most game engines implement custom memory-tracking tools. These tools allow the developers to see how much memory is being used by each engine subsystem and whether or not any memory is leaking (i.e., memory is allocated but never freed). It’s important to have this information so that you can make informed decisions when trying to cut back the memory usage of your game so that it will fit onto the console or type of PC you are targeting.

Keeping track of how much memory a game actually uses can be a surprisingly tricky job. You’d think you could simply wrap `malloc()`/`free()` or `new/delete` in a pair of functions or macros that keep track of the amount of memory that is allocated and freed. However, it’s never that simple for a few reasons:

1. *You often can’t control the allocation behavior of other people’s code.* Unless you write the operating system, drivers and the game engine entirely from scratch, there’s a good chance you’re going to end up linking your game with at least some third-party libraries. Most good libraries provide *memory allocation hooks* so that you can replace their allocators with your own. But some do not. It’s often difficult to keep track of the memory allocated by each and every third-party library you use in your game engine—but it usually *can* be done if you’re thorough and selective in your choice of third-party libraries.
2. *Memory comes in different flavors.* For example, a PC has two kinds of RAM: main RAM and video RAM (the memory residing on your graph-

ics card, which is used primarily for geometry and texture data). Even if you manage to track all of the memory allocations and deallocations occurring within main RAM, it can be well nigh impossible to track video RAM usage. This is because graphics APIs like DirectX actually hide the details of how video RAM is being allocated and used from the developer. On a console, life is a bit easier only because you often end up having to write a video RAM manager yourself. This is more difficult than using DirectX, but at least you have complete knowledge of what's going on.

3. *Allocators come in different flavors.* Many games make use of specialized allocators for various purposes. For example, the Naughty Dog engine has a *global heap* for general-purpose allocations, a special heap for managing the memory created by *game objects* as they spawn into the game world and are destroyed, a *level-loading heap* for data that is *streamed* into memory during gameplay, a stack allocator for *single-frame allocations* (the stack is cleared automatically every frame), an allocator for *video RAM*, and a *debug memory heap* used only for allocations that will not be needed in the final shipping game. Each of these allocators grabs a large hunk of memory when the game starts up and then manages that memory block itself. If we were to track all the calls to new and delete, we'd see one new for each of these six allocators and that's all. To get any useful information, we really need to track all of the allocations *within* each of these allocators' memory blocks.

Most professional game teams expend a significant amount of effort on creating in-engine memory-tracking tools that provide accurate and detailed information. The resulting tools usually provide their output in a variety of forms. For example, the engine might produce a detailed dump of all memory allocations made by the game during a specific period of time. The data might include high water marks for each memory allocator or each game system, indicating the maximum amount of physical RAM required by each. Some engines also provide heads-up displays of memory usage while the game is running. This data might be tabular, as shown in Figure 10.14, or graphical as shown in Figure 10.15.

In addition, when low-memory or out-of-memory conditions arise, a good engine will provide this information in as helpful a way as possible. When PC games are developed, the game team usually works on high-powered PCs with more RAM than the min-spec machine being targeted. Likewise, console games are developed on special *development kits* that have more memory than a retail console. So in both cases, the game can continue to run even when it



Figure 10.14. Tabular memory statistics from Naughty Dog's engine.

Figure 10.15. A graphical memory usage display, also from *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4).

technically has run out of memory (i.e., would no longer fit on a retail console or min-spec PC). When this kind of out-of-memory condition arises, the game engine can display a message saying something like, “Out of memory—this level will not run on a retail system.”

There are lots of other ways in which a game engine’s memory tracking system can aid developers in pinpointing problems as early and as conveniently

as possible. Here are just a few examples:

- If a model fails to load, a bright red text string could be displayed in 3D hovering in the game world where that object would have been.
- If a texture fails to load, the object could be drawn with an ugly pink texture that is very obviously not part of the final game.
- If an animation fails to load, the character could assume a special (possibly humorous) pose that indicates a missing animation, and the name of the missing asset could hover over the character's head.

The key to providing good memory analysis tools is (a) to provide accurate information, (b) to present the data in a way that is convenient and that makes problems obvious and (c) to provide contextual information to aid the team in tracking down the root cause of problems when they occur.

Part III

Graphics, Motion and Sound



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

The Rendering Engine

When most people think about computer and video games, the first thing that comes to mind is the stunning three-dimensional graphics. Real-time 3D rendering is an exceptionally broad and profound topic, so there's simply no way to cover all of the details in a single chapter. Thankfully there are a great many excellent books and other resources available on this topic. In fact, real-time 3D graphics is perhaps one of the best covered of all the technologies that make up a game engine. The goal of this chapter, then, is to provide you with a broad understanding of real-time rendering technology and to serve as a jumping-off point for further learning. After you've read through these pages, you should find that reading other books on 3D graphics seems like a journey through familiar territory. You might even be able to impress your friends at parties (...or alienate them...).

We'll begin by laying a solid foundation in the concepts, theory and mathematics that underlie any real-time 3D rendering engine. Next, we'll have a look at the software and hardware pipelines used to turn this theoretical framework into reality. We'll discuss some of the most common optimization techniques and see how they drive the structure of the tools pipeline and the runtime rendering API in most engines. We'll end with a survey of some of the advanced rendering techniques and lighting models in use by game engines today. Throughout this chapter, I'll point you to some of my favorite books and

other resources that should help you to gain an even deeper understanding of the topics we'll cover here.

11.1 Foundations of Depth-Buffered Triangle Rasterization

When you boil it down to its essence, rendering a three-dimensional scene involves the following basic steps:

- A *virtual scene* is described, usually in terms of 3D surfaces represented in some mathematical form.
- A *virtual camera* is positioned and oriented to produce the desired view of the scene. Typically the camera is modeled as an idealized focal point, with an imaging surface hovering some small distance in front of it, composed of *virtual light sensors* corresponding to the picture elements (*pixels*) of the target display device.
- Various *light sources* are defined. These sources provide all the light rays that will interact with and reflect off the objects in the environment and eventually find their way onto the image-sensing surface of the virtual camera.
- The *visual properties* of the surfaces in the scene are described. This defines how light should interact with each surface.
- For each pixel within the imaging rectangle, the rendering engine calculates the color and intensity of the light ray(s) converging on the virtual camera's focal point through that pixel. This is known as *solving the rendering equation* (also called the *shading equation*).

This high-level rendering process is depicted in Figure 11.1.

Many different technologies can be used to perform the basic rendering steps described above. The primary goal is usually *photorealism*, although some games aim for a more stylized look (e.g., cartoon, charcoal sketch, watercolor and so on). As such, rendering engineers and artists usually attempt to describe the properties of their scenes as realistically as possible and to use light transport models that match physical reality as closely as possible. Within this context, the gamut of rendering technologies ranges from techniques designed for real-time performance at the expense of visual fidelity, to those designed for photorealism but which are not intended to operate in real time.

Real-time rendering engines perform the steps listed above repeatedly, displaying rendered images at a rate of 30, 50 or 60 frames per second to provide

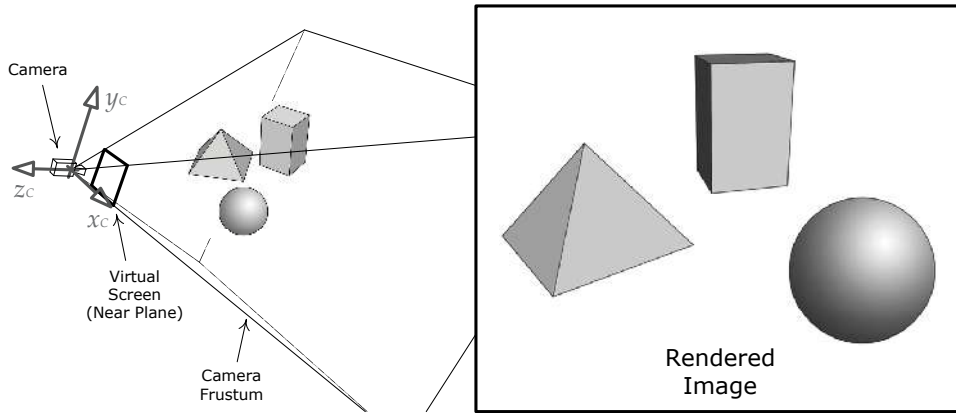


Figure II.1. The high-level rendering approach used by virtually all 3D computer graphics technologies.

the illusion of motion. This means a real-time rendering engine has at most 33.3 ms to generate each image (to achieve a frame rate of 30 FPS). Usually much less time is available, because bandwidth is also consumed by other engine systems like animation, AI, collision detection, physics simulation, audio, player mechanics and other gameplay logic. Considering that film rendering engines often take anywhere from many minutes to many hours to render a single frame, the quality of real-time computer graphics these days is truly astounding.

II.1.1 Describing a Scene

A real-world scene is composed of objects. Some objects are solid, like a brick, and some are amorphous, like a cloud of smoke, but every object occupies a volume of 3D space. An object might be *opaque* (in which case light cannot pass through its volume), *transparent* (in which case light passes through it without being scattered, so that we can see a reasonably clear image of whatever is behind the object), or *translucent* (meaning that light can pass through the object but is scattered in all directions in the process, yielding only a blur of colors that hint at the objects behind it).

Opaque objects can be rendered by considering only their *surfaces*. We don't need to know what's inside an opaque object in order to render it, because light cannot penetrate its surface. When rendering a transparent or translucent object, we really should model how light is reflected, refracted, scattered and absorbed as it passes through the object's volume. This requires knowledge of the interior structure and properties of the object. However,

most game engines don't go to all that trouble. They just render the surfaces of transparent and translucent objects in almost the same way opaque objects are rendered. A simple numeric opacity measure known as *alpha* is used to describe how opaque or transparent a surface is. This approach can lead to various visual anomalies (for example, surface features on the far side of the object may be rendered incorrectly), but the approximation can be made to look reasonably realistic in many cases. Even amorphous objects like clouds of smoke are often represented using particle effects, which are typically composed of large numbers of semitransparent rectangular cards. Therefore, it's safe to say that most game rendering engines are primarily concerned with rendering *surfaces*.

II.I.I.I Representations Used by High-End Rendering Packages

Theoretically, a surface is a two-dimensional sheet comprised of an infinite number of points in three-dimensional space. However, such a description is clearly not practical. In order for a computer to process and render arbitrary surfaces, we need a compact way to represent them numerically.

Some surfaces can be described exactly in analytical form using a *parametric surface equation*. For example, a sphere centered at the origin can be represented by the equation $x^2 + y^2 + z^2 = r^2$. However, parametric equations aren't particularly useful for modeling arbitrary shapes.

In the film industry, surfaces are often represented by a collection of rectangular *patches* each formed from a two-dimensional spline defined by a small number of control points. Various kinds of splines are used, including Bézier surfaces (e.g., bicubic patches, which are third-order Béziers—see http://en.wikipedia.org/wiki/Bezier_surface for more information), nonuniform rational B-splines (NURBS—see <http://en.wikipedia.org/wiki/Nurbs>), Bézier triangles and *N*-patches (also known as *normal patches*—see <http://ubm.io/1iGnvJ5> for more details). Modeling with patches is a bit like covering a statue with little rectangles of cloth or paper maché.

High-end film rendering engines like Pixar's RenderMan use *subdivision surfaces* to define geometric shapes. Each surface is represented by a mesh of control polygons (much like a spline), but the polygons can be subdivided into smaller and smaller polygons using the Catmull-Clark algorithm. This subdivision typically proceeds until the individual polygons are smaller than a single pixel in size. The biggest benefit of this approach is that no matter how close the camera gets to the surface, it can always be subdivided further so that its silhouette edges won't look faceted. To learn more about subdivision surfaces, check out the following great article: <http://ubm.io/1lx6th5>.

11.1.1.2 Triangle Meshes

Game developers have traditionally modeled their surfaces using triangle meshes. Triangles serve as a piecewise linear approximation to a surface, much as a chain of connected line segments acts as a piecewise approximation to a function or curve (see Figure 11.2).

Triangles are the polygon of choice for real-time rendering because they have the following desirable properties:

- *The triangle is the simplest type of polygon.* Any fewer than three vertices, and we wouldn't have a surface at all.
- *A triangle is always planar.* Any polygon with four or more vertices need not have this property because, while the first three vertices define a plane, the fourth vertex might lie above or below that plane.
- *Triangles remain triangles under most kinds of transformations, including affine transforms and perspective projections.* At worst, a triangle viewed edge-on will degenerate into a line segment. At every other orientation, it remains triangular.
- *Virtually all commercial graphics-acceleration hardware is designed around triangle rasterization.* Starting with the earliest 3D graphics accelerators for the PC, rendering hardware has been designed almost exclusively around triangle rasterization. This decision can be traced all the way back to the first software rasterizers used in the earliest 3D games like *Castle Wolfenstein 3D* and *Doom*. Like it or not, triangle-based technologies are entrenched in our industry and probably will be for years to come.

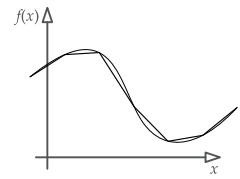


Figure 11.2. A mesh of triangles is a linear approximation to a surface, just as a series of connected line segments can serve as a linear approximation to a function or curve.

Tessellation

The term *tessellation* describes a process of dividing a surface up into a collection of discrete polygons (which are usually either quadrilaterals, also known as *quads*, or triangles). *Triangulation* is tessellation of a surface into triangles.

One problem with the kind of triangle mesh used in games is that its level of tessellation is fixed by the artist when he or she creates it. Fixed tessellation can cause an object's *silhouette edges* to look blocky, as shown in Figure 11.3; this is especially noticeable when the object is close to the camera.

Ideally, we'd like a solution that can arbitrarily increase tessellation as an object gets closer to the virtual camera. In other words, we'd like to have a uniform triangle-to-pixel density, no matter how close or far away the object is. Subdivision surfaces can achieve this ideal—surfaces can be tessellated based on distance from the camera, so that every triangle is less than one pixel in size.

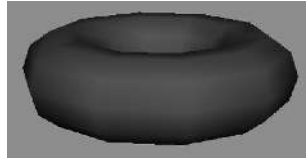


Figure II.3. Fixed tessellation can cause an object's silhouette edges to look blocky, especially when the object is close to the camera.

Game developers often attempt to approximate this ideal of uniform triangle-to-pixel density by creating a chain of alternate versions of each triangle mesh, each known as a *level of detail* (LOD). The first LOD, often called LOD 0, represents the highest level of tessellation; it is used when the object is very close to the camera. Subsequent LODs are tessellated at lower and lower resolutions (see Figure 11.4). As the object moves farther away from the camera, the engine switches from LOD 0 to LOD 1 to LOD 2 and so on. This allows the rendering engine to spend the majority of its time transforming and lighting the vertices of the objects that are closest to the camera (and therefore occupy the largest number of pixels on-screen).

Some game engines apply *dynamic tessellation* techniques to expansive meshes like water or terrain. In this technique, the mesh is usually represented by a height field defined on some kind of regular grid pattern. The region of the mesh that is closest to the camera is tessellated to the full resolution of the grid. Regions that are farther away from the camera are tessellated using fewer and fewer grid points.

Progressive meshes are another technique for dynamic tessellation and LOD-ing. With this technique, a single high-resolution mesh is created for display when the object is very close to the camera. (This is essentially the

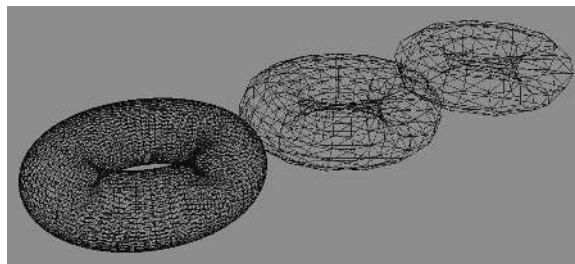


Figure II.4. A chain of LOD meshes, each with a fixed level of tessellation, can be used to approximate uniform triangle-to-pixel density. The leftmost torus is constructed from 5000 triangles, the center torus from 450 triangles and the rightmost torus from 200 triangles.

LOD 0 mesh.) This mesh is automatically detessellated as the object gets farther away by collapsing certain edges. In effect, this process automatically generates a semi-continuous chain of LODs. See <http://research.microsoft.com/en-us/um/people/hoppe/pm.pdf> for a detailed discussion of progressive mesh technology.

11.1.1.3 Constructing a Triangle Mesh

Now that we understand what triangle meshes are and why they're used, let's take a brief look at how they're constructed.

Winding Order

A triangle is defined by the position vectors of its three vertices, which we can denote $\mathbf{p}_1$, $\mathbf{p}_2$ and $\mathbf{p}_3$. The edges of a triangle can be found by simply subtracting the position vectors of adjacent vertices. For example,

$$\begin{aligned}\mathbf{e}_{12} &= \mathbf{p}_2 - \mathbf{p}_1, \\ \mathbf{e}_{13} &= \mathbf{p}_3 - \mathbf{p}_1, \\ \mathbf{e}_{23} &= \mathbf{p}_3 - \mathbf{p}_2.\end{aligned}$$

The normalized cross product of any two edges defines a unit *face normal* $\mathbf{N}$:

$$\mathbf{N} = \frac{\mathbf{e}_{12} \times \mathbf{e}_{13}}{|\mathbf{e}_{12} \times \mathbf{e}_{13}|}.$$

These derivations are illustrated in Figure 11.5. To know the *direction* of the face normal (i.e., the sense of the edge cross product), we need to define which side of the triangle should be considered the front (i.e., the outside surface of an object) and which should be the back (i.e., its inside surface). This can be defined easily by specifying a *winding order*—clockwise (CW) or counterclockwise (CCW).

Most low-level graphics APIs give us a way to *cull* back-facing triangles based on winding order. For example, if we set the cull mode parameter in

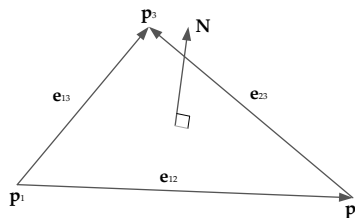


Figure 11.5. Deriving the edges and plane of a triangle from its vertices.

Direct3D (D3DRS_CULL) to D3DCULLMODE_CW, then any triangle whose vertices wind in a clockwise fashion in screen space will be treated as a back-facing triangle and will not be drawn.

Back-face culling is important because we generally don't want to waste time drawing triangles that aren't going to be visible anyway. Also, rendering the back faces of transparent objects can actually cause visual anomalies. The choice of winding order is an arbitrary one, but of course it must be consistent across all assets in the entire game. Inconsistent winding order is a common error among junior 3D modelers.

Triangle Lists

The easiest way to define a mesh is simply to list the vertices in groups of three, each triple corresponding to a single triangle. This data structure is known as a *triangle list*; it is illustrated in Figure 11.6.

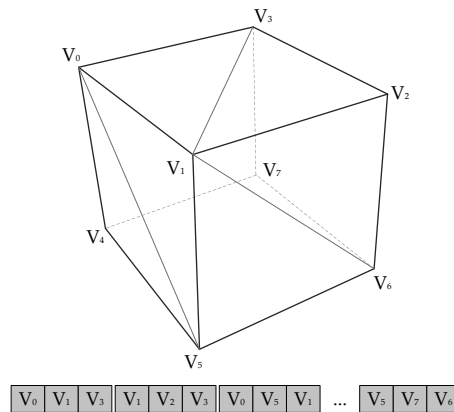


Figure 11.6. A triangle list.

Indexed Triangle Lists

You probably noticed that many of the vertices in the triangle list shown in Figure 11.6 were duplicated, often multiple times. As we'll see in Section 11.1.2.1, we often store quite a lot of metadata with each vertex, so repeating this data in a triangle list wastes memory. It also wastes GPU bandwidth, because a duplicated vertex will be transformed and lit multiple times.

For these reasons, most rendering engines make use of a more efficient data structure known as an *indexed triangle list*. The basic idea is to list the vertices once with no duplication and then to use lightweight vertex *indices* (usually occupying only 16 bits each) to define the triples of vertices that constitute the

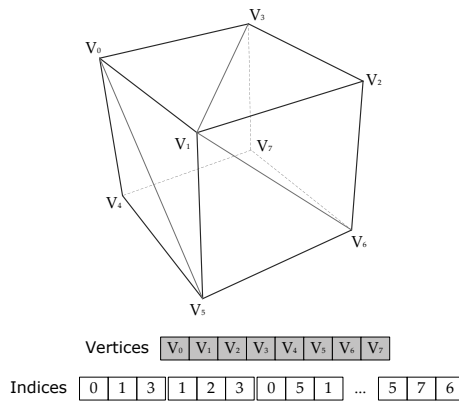


Figure 11.7. An indexed triangle list.

triangles. The vertices are stored in an array known as a *vertex buffer* (DirectX) or *vertex array* (OpenGL). The indices are stored in a separate buffer known as an *index buffer* or *index array*. This technique is shown in Figure 11.7.

Strips and Fans

Specialized mesh data structures known as *triangle strips* and *triangle fans* are sometimes used for game rendering. Both of these data structures eliminate the need for an index buffer, while still reducing vertex duplication to some degree. They accomplish this by predefining the order in which vertices must appear and how they are combined to form triangles.

In a strip, the first three vertices define the first triangle. Each subsequent vertex forms an entirely new triangle, along with its previous two neighbors. To keep the winding order of a triangle strip consistent, the previous two neighbor vertices swap places after each new triangle. A triangle strip is shown in Figure 11.8.

In a fan, the first three vertices define the first triangle and each subsequent vertex defines a new triangle with the previous vertex and the first vertex in the fan. This is illustrated in Figure 11.9.

Vertex Cache Optimization

When a GPU processes an indexed triangle list, each triangle can refer to any vertex within the vertex buffer. The vertices must be processed in the order they appear within the triangles, because the integrity of each triangle must be maintained for the rasterization stage. As vertices are processed by the vertex shader, they are cached for reuse. If a subsequent primitive refers to a vertex

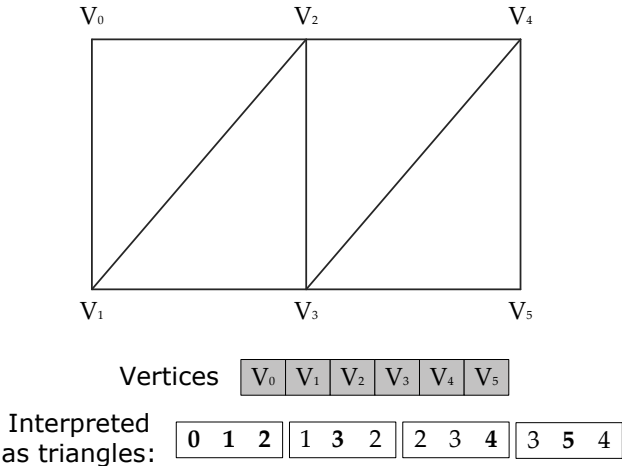


Figure II.8. A triangle strip.

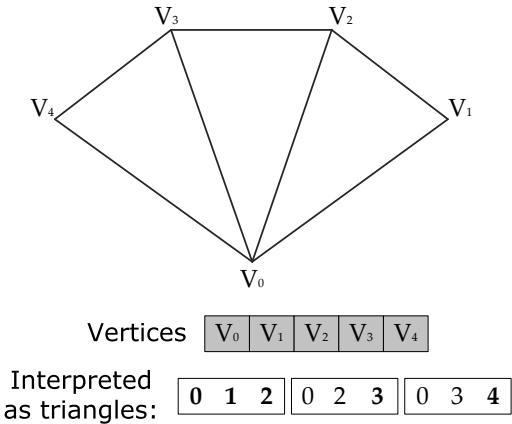


Figure II.9. A triangle fan.

that already resides in the cache, its processed attributes are used instead of reprocessing the vertex.

Strips and fans are used in part because they can potentially save memory (no index buffer required) and in part because they tend to improve the cache coherency of the memory accesses made by the GPU to video RAM. Even better, we can use an *indexed strip* or *indexed fan* to virtually eliminate vertex duplication (which can often save more memory than eliminating the index buffer), while still reaping the cache coherency benefits of the strip or fan vertex ordering.

Indexed triangle lists can also be cache-optimized without restricting ourselves to strip or fan vertex ordering. A *vertex cache optimizer* is an offline geometry processing tool that attempts to list the triangles in an order that optimizes vertex reuse within the cache. It generally takes into account factors such as the size of the vertex cache(s) present on a particular type of GPU and the algorithms used by the GPU to decide when to cache vertices and when to discard them. For example, the vertex cache optimizer included in Sony’s *Edge* geometry processing library can achieve rendering throughput that is up to 4% better than what is possible with triangle stripping.

II.I.I.4 Model Space

The position vectors of a triangle mesh’s vertices are usually specified relative to a convenient local coordinate system called *model space*, *local space*, or *object space*. The origin of model space is usually either in the center of the object or at

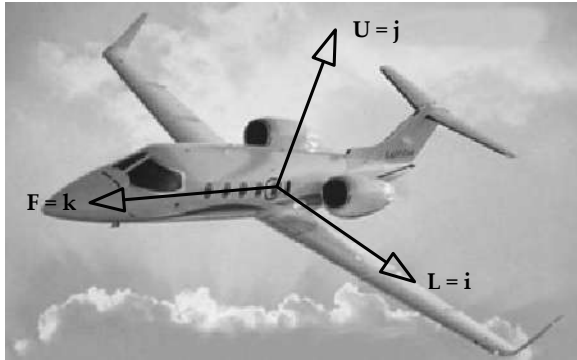


Figure 11.10. One possible mapping of the model-space axes.

some other convenient location, like on the floor between the feet of a character or on the ground at the horizontal centroid of the wheels of a vehicle.

As we learned in Section 5.3.9.1, the sense of the model-space axes is arbitrary, but the axes typically align with the natural “front,” “left,” “right” and “up” directions on the model. For a little mathematical rigor, we can define three unit vectors F , L (or R) and U and map them as desired onto the unit basis vectors i , j and k (and hence to the x -, y - and z -axes, respectively) in model space. For example, a common mapping is $L = i$, $U = j$ and $F = k$. The mapping is completely arbitrary, but it’s important to be consistent for all models across the entire engine. Figure 11.10 shows one possible mapping of the model-space axes for an aircraft model.

11.1.1.5 World Space and Mesh Instancing

Many individual meshes are composed into a complete scene by positioning and orienting them within a common coordinate system known as *world space*. Any one mesh might appear many times in a scene—examples include a street lined with identical lamp posts, a faceless mob of soldiers or a swarm of spiders attacking the player. We call each such object a *mesh instance*.

A mesh instance contains a reference to its shared mesh data and also includes a transformation matrix that converts the mesh’s vertices from model space to world space, within the context of that particular instance. This matrix is called the *model-to-world* matrix, or sometimes just the *world matrix*. Using the notation from Section 5.3.10.2, this matrix can be written as follows:

$$\mathbf{M}_{M \rightarrow W} = \begin{bmatrix} (\mathbf{RS})_{M \rightarrow W} & 0 \\ \mathbf{t}_M & 1 \end{bmatrix},$$

where the upper 3×3 matrix $(\mathbf{RS})_{M \rightarrow W}$ rotates and scales model-space vertices into world space, and $\mathbf{t}_M$ is the translation of the model-space axes expressed in world space. If we have the unit model-space basis vectors $\mathbf{i}_M$, $\mathbf{j}_M$ and $\mathbf{k}_M$, expressed in world-space coordinates, this matrix can also be written as follows:

$$\mathbf{M}_{M \rightarrow W} = \left[\begin{array}{c|c} \mathbf{i}_M & 0 \\ \mathbf{j}_M & 0 \\ \mathbf{k}_M & 0 \\ \hline \mathbf{t}_M & 1 \end{array} \right].$$

Given a vertex expressed in model-space coordinates, the rendering engine calculates its world-space equivalent as follows:

$$\mathbf{v}_W = \mathbf{v}_M \mathbf{M}_{M \rightarrow W}.$$

We can think of the matrix $\mathbf{M}_{M \rightarrow W}$ as a description of the position and orientation of the model-space axes themselves, expressed in world-space coordinates. Or we can think of it as a matrix that transforms vertices from model space to world space.

When rendering a mesh, the model-to-world matrix is also applied to the surface normals of the mesh (see Section 11.1.2.1). Recall from Section 5.3.11, that in order to transform normal vectors properly, we must multiply them by the inverse transpose of the model-to-world matrix. If our matrix does not contain any scale or shear, we can transform our normal vectors correctly by simply setting their w components to zero prior to multiplication by the model-to-world matrix, as described in Section 5.3.6.1.

Some meshes like buildings, terrain and other background elements are entirely static and unique. The vertices of these meshes are often expressed in world space, so their model-to-world matrices are identity and can be ignored.

11.1.2 Describing the Visual Properties of a Surface

In order to properly render and light a surface, we need a description of its *visual properties*. Surface properties include geometric information, such as the direction of the surface normal at various points on the surface. They also encompass a description of how light should interact with the surface. This includes diffuse color, shininess/reflectivity, roughness or texture, degree of opacity or transparency, index of refraction and other optical properties. Surface properties might also include a specification of how the surface should change over time (e.g., how an animated character's skin should track the joints of its skeleton or how the surface of a body of water should move).

The key to rendering photorealistic images is properly accounting for light's behavior as it interacts with the objects in the scene. Hence rendering engineers need to have a good understanding of how light works, how it is transported through an environment and how the virtual camera "senses" it and translates it into the colors stored in the pixels on-screen.

11.1.2.1 Introduction to Light and Color

Light is electromagnetic radiation; it acts like both a wave and a particle in different situations. The color of light is determined by its *intensity* I and its *wavelength* λ (or its frequency f , where $f = 1/\lambda$). The visible gamut ranges from a wavelength of 740 nm (or a frequency of 430 THz) to a wavelength of 380 nm (750 THz). A beam of light may contain a single pure wavelength (i.e., the colors of the rainbow, also known as the *spectral colors*), or it may contain a mixture of various wavelengths. We can draw a graph showing how much of each frequency a given beam of light contains, called a *spectral plot*. White light contains a little bit of all wavelengths, so its spectral plot would look roughly like a box extending across the entire visible band. Pure green light contains only one wavelength, so its spectral plot would look like a single infinitesimally narrow spike at about 570 THz.

Light-Object Interactions

Light can have many complex interactions with matter. Its behavior is governed in part by the *medium* through which it is traveling and in part by the shape and properties of the *interfaces* between different types of media (air-solid, air-water, water-glass, etc.). Technically speaking, a surface is really just an interface between two different types of media.

Despite all of its complexity, light can really only do four things:

- It can be *absorbed*.
- It can be *reflected*.
- It can be *transmitted* through an object, usually being *refracted* (bent) in the process.
- It can be *diffracted* when passing through very narrow openings.

Most photorealistic rendering engines account for the first three of these behaviors; diffraction is not usually taken into account because its effects are rarely noticeable in most scenes.

Only certain wavelengths may be absorbed by a surface, while others are reflected. This is what gives rise to our perception of the color of an object. For example, when white light falls on a red object, all wavelengths except

red are absorbed, hence the object appears red. The same perceptual effect is achieved when red light is cast onto a white object—our eyes don't know the difference.

Reflections can be *diffuse*, meaning that an incoming ray is scattered equally in all directions. Reflections can also be *specular*, meaning that an incident light ray will reflect directly or be spread only into a narrow cone. Reflections can also be *anisotropic*, meaning that the way in which light reflects from a surface changes depending on the angle at which the surface is viewed.

When light is transmitted through a volume, it can be *scattered* (as is the case for translucent objects), partially *absorbed* (as with colored glass), or *refracted* (as happens when light travels through a prism). The refraction angles can be different for different wavelengths, leading to spectral spreading. This is why we see rainbows when light passes through raindrops and glass prisms. Light can also enter a semi-solid surface, bounce around and then exit the surface at a different point from the one at which it entered the surface. We call this *subsurface scattering*, and it is one of the effects that gives skin, wax and marble their characteristic warm appearance.

Color Spaces and Color Models

A *color model* is a three-dimensional coordinate system that measures colors. A *color space* is a specific standard for how numerical colors in a particular color model should be mapped onto the colors perceived by human beings in the real world. Color models are typically three-dimensional because of the three types of color sensors (cones) in our eyes, which are sensitive to different wavelengths of light.

The most commonly used color model in computer graphics is the RGB model. In this model, color space is represented by a unit cube, with the relative intensities of red, green and blue light measured along its axes. The red, green and blue components are called *color channels*. In the canonical RGB color model, each channel ranges from zero to one. So the color (0, 0, 0) represents black, while (1, 1, 1) represents white.

When colors are stored in a bitmapped image, various color formats can be employed. A color format is defined in part by the number of *bits per pixel* it occupies and, more specifically, the number of bits used to represent each color channel. The RGB888 format uses eight bits per channel, for a total of 24 bits per pixel. In this format, each channel ranges from 0 to 255 rather than from zero to one. RGB565 uses five bits for red and blue and six for green, for a total of 16 bits per pixel. A paletted format might use eight bits per pixel to store indices into a 256-element color palette, each entry of which might be stored in RGB888 or some other suitable format.

A number of other color models are also used in 3D rendering. We'll see how the log-LUV color model is used for *high dynamic range* (HDR) lighting in Section 11.3.1.5.

Opacity and the Alpha Channel

A fourth channel called *alpha* is often tacked on to RGB color vectors. As mentioned in Section 11.1.1, alpha measures the opacity of an object. When stored in an image pixel, alpha represents the opacity of the pixel.

RGB color formats can be extended to include an alpha channel, in which case they are referred to as RGBA or ARGB color formats. For example, RGBA8888 is a 32 bit-per-pixel format with eight bits each for red, green, blue and alpha. RGBA5551 is a 16 bit-per-pixel format with one-bit alpha; in this format, colors can either be fully opaque or fully transparent.

11.1.2.2 Vertex Attributes

The simplest way to describe the visual properties of a surface is to specify them at discrete points on the surface. The vertices of a mesh are a convenient place to store surface properties, in which case they are called *vertex attributes*.

A typical triangle mesh includes some or all of the following attributes at each vertex. As rendering engineers, we are of course free to define any additional attributes that may be required in order to achieve a desired visual effect on-screen.

- *Position vector* ($\mathbf{p}_i = [p_{ix} \ p_{iy} \ p_{iz}]$). This is the 3D position of the i th vertex in the mesh. It is usually specified in a coordinate space local to the object, known as *model space*.
- *Vertex normal* ($\mathbf{n}_i = [n_{ix} \ n_{iy} \ n_{iz}]$). This vector defines the unit surface normal at the position of vertex i . It is used in per-vertex dynamic lighting calculations.
- *Vertex tangent* ($\mathbf{t}_i = [t_{ix} \ t_{iy} \ t_{iz}]$) and *bitangent* ($\mathbf{b}_i = [b_{ix} \ b_{iy} \ b_{iz}]$). These two unit vectors lie perpendicular to one another and to the vertex normal $\mathbf{n}_i$. Together, the three vectors $\mathbf{n}_i$, $\mathbf{t}_i$ and $\mathbf{b}_i$ define a set of coordinate axes known as *tangent space*. This space is used for various per-pixel lighting calculations, such as normal mapping and environment mapping. (The bitangent $\mathbf{b}_i$ is sometimes confusingly called the *binormal*, even though it is *not* normal to the surface.)
- *Diffuse color* ($\mathbf{d}_i = [d_{iR} \ d_{iG} \ d_{iB} \ d_{iA}]$). This four-element vector describes the diffuse color of the surface, expressed in the RGB color space. It typically also includes a specification of the opacity or *alpha* (A) of the

surface at the position of the vertex. This color may be calculated offline (static lighting) or at runtime (dynamic lighting).

- *Specular color* ($\mathbf{s}_i = [s_{iR} \ s_{iG} \ s_{iB} \ s_{iA}]$). This quantity describes the color of the specular highlight that should appear when light reflects directly from a shiny surface onto the virtual camera's imaging plane.
- *Texture coordinates* ($\mathbf{u}_{ij} = [u_{ij} \ v_{ij}]$). Texture coordinates allow a two- (or sometimes three-) dimensional bitmap to be “shrink wrapped” onto the surface of a mesh—a process known as *texture mapping*. A texture coordinate (u, v) describes the location of a particular vertex within the two-dimensional normalized coordinate space of the texture. A triangle can be mapped with more than one texture; hence it can have more than one set of texture coordinates. We've denoted the distinct sets of texture coordinates via the subscript j above.
- *Skinning weights* ($\mathbf{k}_{ij} = [k_{ij} \ w_{ij}]$). In skeletal animation, the vertices of a mesh are attached to individual joints in an articulated skeleton. In this case, each vertex must specify to which joint it is attached via an index, k . A vertex can be influenced by multiple joints, in which case the final vertex position becomes a *weighted average* of these influences. Thus, the weight of each joint's influence is denoted by a weighting factor w . In general, a vertex i can have multiple joint influences j , each denoted by the pair of numbers (k_{ij}, w_{ij}) .

II.1.2.3 Vertex Formats

Vertex attributes are typically stored within a data structure such as a C struct or a C++ class. The layout of such a data structure is known as a *vertex format*. Different meshes require different combinations of attributes and hence need different vertex formats. The following are some examples of common vertex formats:

```
// Simplest possible vertex -- position only (useful for
// shadow volume extrusion, silhouette edge detection
// for cartoon rendering, z-prepass, etc.)
struct Vertex1P
{
    Vector3  m_p;  // position
};

// A typical vertex format with position, vertex normal
// and one set of texture coordinates.
struct Vertex1P1N1UV
{
    Vector3 m_p;    // position
```

```

    Vector3 m_n;        // vertex normal
    F32      m_uv[2];    // (u, v) texture coordinate
};

// A skinned vertex with position, diffuse and specular
// colors and four weighted joint influences.
struct Vertex1P1D1S2UV4J
{
    Vector3 m_p;        // position
    Color4  m_d;        // diffuse color and translucency
    Color4  m_s;        // specular color
    F32      m_uv0[2];  // first set of tex coords
    F32      m_uv1[2];  // second set of tex coords
    U8       m_k[4];    // four joint indices, and...
    F32      m_w[3];    // three joint weights, for
                        // skinning (fourth is calc'd
                        // from the first three)
};

```

Clearly the number of possible permutations of vertex attributes—and hence the number of distinct vertex formats—can grow to be extremely large. (In fact the number of formats is theoretically unbounded, if one were to permit any number of texture coordinates and/or joint weights.) Management of all these vertex formats is a common source of headaches for any graphics programmer.

Some steps can be taken to reduce the number of vertex formats that an engine has to support. In practical graphics applications, many of the theoretically possible vertex formats are simply not useful, or they cannot be handled by the graphics hardware or the game’s shaders. Some game teams also limit themselves to a subset of the useful/feasible vertex formats in order to keep things more manageable. For example, they might only allow zero, two or four joint weights per vertex, or they might decide to support no more than two sets of texture coordinates per vertex. Some GPUs are capable of extracting a subset of attributes from a vertex data structure, so game teams can also choose to use a single “überformat” for all meshes and let the hardware select the relevant attributes based on the requirements of the shader.

11.1.2.4 Attribute Interpolation

The attributes at a triangle’s vertices are just a coarse, discretized approximation to the visual properties of the surface as a whole. When rendering a triangle, what really matters are the visual properties at the interior points of the triangle as “seen” through each pixel on-screen. In other words, we need to know the values of the attributes on a *per-pixel* basis, not a per-vertex basis.

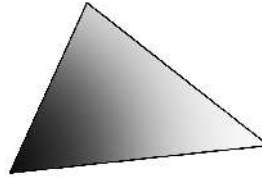


Figure II.11. A Gouraud-shaded triangle with different shades of gray at the vertices.

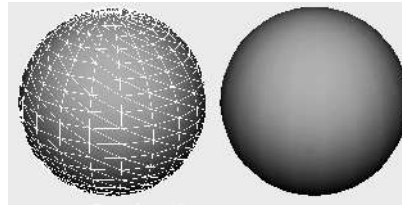


Figure II.12. Gouraud shading can make faceted objects appear to be smooth.

One simple way to determine the per-pixel values of a mesh's surface attributes is to *linearly interpolate* the per-vertex attribute data. When applied to vertex colors, attribute interpolation is known as *Gouraud shading*. An example of Gouraud shading applied to a triangle is shown in Figure 11.11, and its effects on a simple triangle mesh are illustrated in Figure 11.12. Interpolation is routinely applied to other kinds of vertex attribute information as well, such as vertex normals, texture coordinates and depth.

Vertex Normals and Smoothing

As we'll see in Section 11.1.3, *lighting* is the process of calculating the color of an object at various points on its surface, based on the visual properties of the surface and the properties of the light impinging upon it. The simplest way to light a mesh is to calculate the color of the surface on a *per-vertex* basis. In other words, we use the properties of the surface and the incoming light to calculate the diffuse color of each vertex ($\mathbf{d}_i$). These vertex colors are then interpolated across the triangles of the mesh via Gouraud shading.

In order to determine how a ray of light will reflect from a point on a surface, most lighting models make use of a vector that is *normal* to the surface at the point of the light ray's impact. Since we're performing lighting calculations on a per-vertex basis, we can use the vertex normal $\mathbf{n}_i$ for this purpose. Therefore, the directions of a mesh's vertex normals can have a significant impact on the final appearance of a mesh.

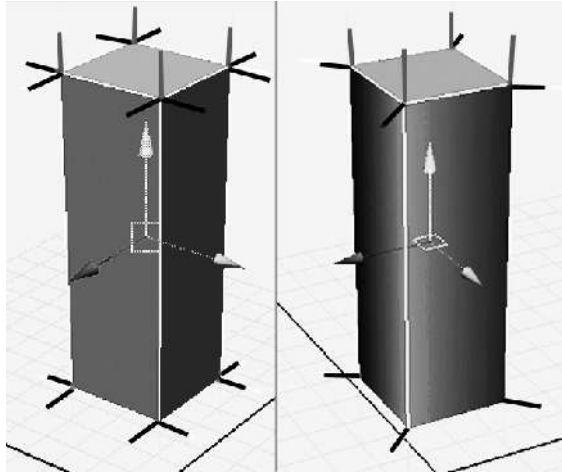


Figure 11.13. The directions of a mesh’s vertex normals can have a profound effect on the colors calculated during per-vertex lighting calculations.

As an example, consider a tall, thin, four-sided box. If we want the box to appear to be sharp-edged, we can specify the vertex normals to be perpendicular to the faces of the box. As we light each triangle, we will encounter the same normal vector at all three vertices, so the resulting lighting will appear flat, and it will abruptly change at the corners of the box just as the vertex normals do.

We can also make the same box mesh look a bit like a smooth cylinder by specifying vertex normals that point radially outward from the box’s center line. In this case, the vertices of each triangle will have different vertex normals, causing us to calculate different colors at each vertex. Gouraud shading will smoothly interpolate these vertex colors, resulting in lighting that appears to vary smoothly across the surface. This effect is illustrated in Figure 11.13.

11.1.2.5 Textures

When triangles are relatively large, specifying surface properties on a per-vertex basis can be too coarse-grained. Linear attribute interpolation isn’t always what we want, and it can lead to undesirable visual anomalies.

As an example, consider the problem of rendering the bright *specular highlight* that can occur when light shines on a glossy object. If the mesh is highly tessellated, per-vertex lighting combined with Gouraud shading can yield reasonably good results. However, when the triangles are too large, the errors that arise from linearly interpolating the specular highlight can become jarringly obvious, as shown in Figure 11.14.

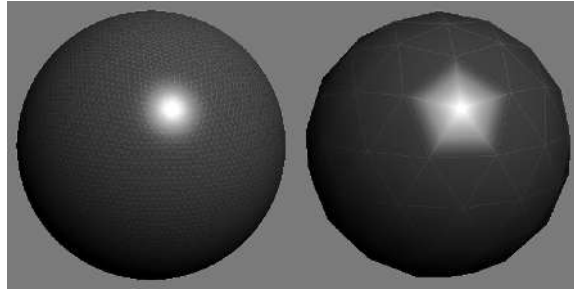


Figure II.14. Linear interpolation of vertex attributes does not always yield an adequate description of the visual properties of a surface, especially when tessellation is low.

To overcome the limitations of per-vertex surface attributes, rendering engineers use bitmapped images known as *texture maps*. A texture often contains color information and is usually projected onto the triangles of a mesh. In this case, it acts a bit like those silly fake tattoos we used to apply to our arms when we were kids. But a texture can contain other kinds of visual surface properties as well as colors. And a texture needn't be projected onto a mesh—for example, a texture might be used as a stand-alone data table. The individual picture elements of a texture are called *texels* to differentiate them from the pixels on the screen.

The dimensions of a texture bitmap are constrained to be powers of two on some graphics hardware. Typical texture dimensions include 256×256 , 512×512 , 1024×1024 and 2048×2048 , although textures can be any size on most hardware, provided the texture fits into video memory. Some graphics hardware imposes additional restrictions, such as requiring textures to be square, or lifts some restrictions, such as not constraining texture dimensions to be powers of two.

Types of Textures

The most common type of texture is known as a *diffuse map*, or *albedo map*. It describes the diffuse surface color at each texel on a surface and acts like a decal or paint job on the surface.

Other types of textures are used in computer graphics as well, including *normal maps* (which store unit normal vectors at each texel, encoded as RGB values), *gloss maps* (which encode how shiny a surface should be at each texel), *environment maps* (which contain a picture of the surrounding environment for rendering reflections) and many others. See Section 11.3.1 for a discussion of how various types of textures can be used for image-based lighting and other effects.

We can actually use texture maps to store any information that we happen to need in our lighting calculations. For example, a one-dimensional texture could be used to store sampled values of a complex math function, a color-to-color mapping table, or any other kind of look-up table (LUT).

Texture Coordinates

Let's consider how to project a two-dimensional texture onto a mesh. To do this, we define a two-dimensional coordinate system known as *texture space*. A texture coordinate is usually represented by a normalized pair of numbers denoted (u, v) . These coordinates always range from $(0, 0)$ at the bottom left corner of the texture to $(1, 1)$ at the top right. Using normalized coordinates like this allows the same coordinate system to be used regardless of the dimensions of the texture.

To map a triangle onto a 2D texture, we simply specify a pair of texture coordinates (u_i, v_i) at each vertex i . This effectively maps the triangle onto the image plane in texture space. An example of texture mapping is depicted in Figure 11.15.

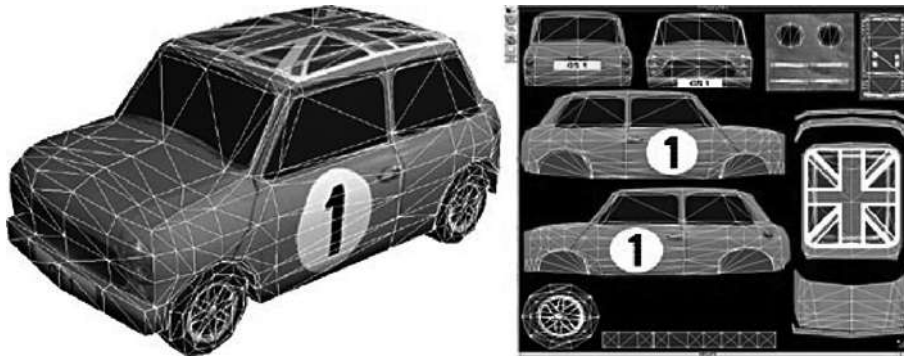


Figure 11.15. An example of texture mapping. The triangles are shown both in three-dimensional space and in texture space.

Texture Addressing Modes

Texture coordinates are permitted to extend beyond the $[0, 1]$ range. The graphics hardware can handle out-of-range texture coordinates in any one of the following ways. These are known as *texture addressing modes*; which mode is used is under the control of the user.

- *Wrap*. In this mode, the texture is repeated over and over in every direction. All texture coordinates of the form (ju, kv) are equivalent to the coordinate (u, v) , where j and k are arbitrary integers.

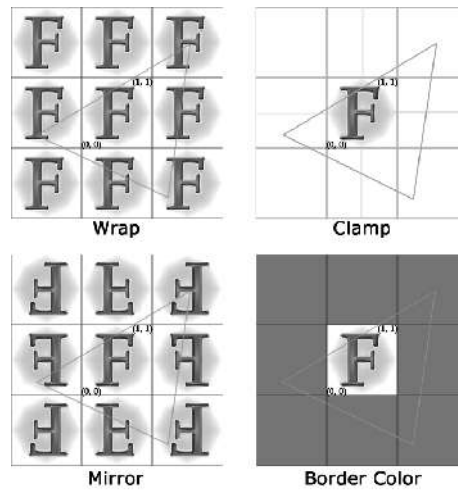


Figure 11.16. Texture addressing modes.

- *Mirror*. This mode acts like wrap mode, except that the texture is mirrored about the v -axis for odd integer multiples of u , and about the u -axis for odd integer multiples of v .
- *Clamp*. In this mode, the colors of the texels around the outer edge of the texture are simply extended when texture coordinates fall outside the normal range.
- *Border color*. In this mode, an arbitrary user-specified color is used for the region outside the $[0, 1]$ texture coordinate range.

These texture addressing modes are depicted in Figure 11.16.

Texture Formats

Texture bitmaps can be stored on disk in virtually any image format, provided your game engine includes the code necessary to read it into memory. Common formats include Targa (.tga), Portable Network Graphics (.png), Windows Bitmap (.bmp) and Tagged Image File Format (.tif). In memory, textures are usually represented as two-dimensional (strided) arrays of pixels using various color formats, including RGB888, RGBA8888, RGB565, RGBA5551 and so on.

Most modern graphics cards and graphics APIs support *compressed textures*. DirectX supports a family of compressed formats known as DXT or S3 Texture Compression (S3TC). We won't cover the details here, but the basic idea is to break the texture into 4×4 blocks of pixels and use a small

color palette to store the colors for each block. You can read more about S3 compressed texture formats at http://en.wikipedia.org/wiki/S3_Texture_Compression.

Compressed textures have the obvious benefit of using less memory than their uncompressed counterparts. An additional unexpected plus is that they are faster to render with as well. S3 compressed textures achieve this speed-up because of more cache-friendly memory access patterns— 4×4 blocks of adjacent pixels are stored in a single 64- or 128-bit machine word—and because more of the texture can fit into the cache at once. Compressed textures do suffer from compression artifacts. While the anomalies are usually not noticeable, there are situations in which uncompressed textures must be used.

Texel Density and Mipmapping

Imagine rendering a full-screen quad (a rectangle composed of two triangles) that has been mapped with a texture whose resolution exactly matches that of the screen. In this case, each texel maps exactly to a single pixel on-screen, and we say that the *texel density* (ratio of texels to pixels) is one. When this same quad is viewed at a distance, its on-screen area becomes smaller. The resolution of the texture hasn't changed, so the quad's texel density is now greater than one (meaning that more than one texel is contributing to each pixel).

Clearly texel density is not a fixed quantity—it changes as a texture-mapped object moves relative to the camera. Texel density affects the memory consumption and the visual quality of a three-dimensional scene. When the texel density is much less than one, the texels become significantly larger than a pixel on-screen, and you can start to see the edges of the texels. This destroys the illusion. When texel density is much greater than one, many texels contribute to a single pixel on-screen. This can cause a *moiré banding pattern*, as shown in Figure 11.17. Worse, a pixel's color can appear to swim and flicker as different texels within the boundaries of the pixel dominate its color depending on subtle changes in camera angle or position. Rendering a distant object with a very high texel density can also be a waste of memory if the player can *never* get close to it. After all, why keep such a high-res texture in memory if no one will ever see all that detail?

Ideally we'd like to maintain a texel density that is close to one at all times, for both nearby and distant objects. This is impossible to achieve exactly, but it can be approximated via a technique called *mipmapping*. For each texture, we create a sequence of lower-resolution bitmaps, each of which is one-half the width and one-half the height of its predecessor. We call each of these images a *mipmap*, or *mip level*. For example, a 64×64 texture would have the

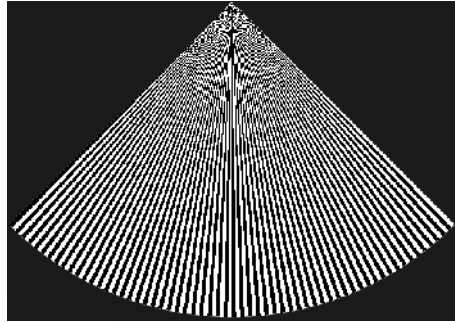


Figure II.17. A texel density greater than one can lead to a moiré pattern.

following mip levels: 64×64 , 32×32 , 16×16 , 8×8 , 4×4 , 2×2 and 1×1 , as shown in Figure 11.18. Once we have mipmapped our textures, the graphics hardware selects the appropriate mip level based on a triangle's distance away from the camera, in an attempt to maintain a texel density that is close to one. For example, if a texture takes up an area of 40×40 on-screen, the 64×64 mip level might be selected; if that same texture takes up only a 10×10 area, the 16×16 mip level might be used. As we'll see below, *trilinear filtering* allows the hardware to sample two adjacent mip levels and blend the results. In this case, a 10×10 area might be mapped by blending the 16×16 and 8×8 mip levels together.

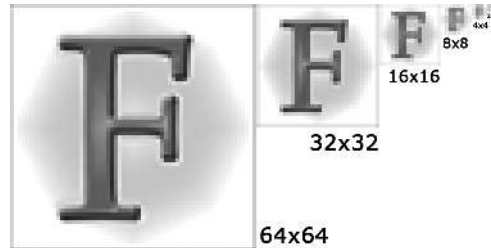


Figure II.18. Mip levels for a 64×64 texture.

World-Space Texel Density

The term “texel density” can also be used to describe the ratio of texels to world-space area on a textured surface. For example, a 2 m cube mapped with a 256×256 texture would have a texel density of $256^2 / 2^2 = 16,384$. I will call this *world-space texel density* to differentiate it from the screen-space texel density we’ve been discussing thus far.

World-space texel density need not be close to one, and in fact the specific value will usually be much greater than one and depends entirely upon your choice of world units. Nonetheless, it is important for objects to be texture mapped with a reasonably consistent world-space texel density. For example, we would expect all six sides of a cube to occupy the same texture area. If this were not the case, the texture on one side of the cube would have a lower-resolution appearance than another side, which can be noticeable to the player. Many game studios provide their art teams with guidelines and in-engine texel density visualization tools in an effort to ensure that all objects in the game have a reasonably consistent world-space texel density.

Texture Filtering

When rendering a pixel of a textured triangle, the graphics hardware samples the texture map by considering where the pixel center falls in texture space. There is usually not a clean one-to-one mapping between texels and pixels, and pixel centers can fall at any place in texture space, including directly on the boundary between two or more texels. Therefore, the graphics hardware usually has to sample more than one texel and blend the resulting colors to arrive at the actual sampled texel color. We call this *texture filtering*.

Most graphics cards support the following kinds of texture filtering:

- *Nearest neighbor*. In this crude approach, the texel whose center is closest to the pixel center is selected. When mipmapping is enabled, the mip level is selected whose resolution is nearest to but greater than the ideal theoretical resolution needed to achieve a screen-space texel density of one.
- *Bilinear*. In this approach, the four texels surrounding the pixel center are sampled, and the resulting color is a weighted average of their colors (where the weights are based on the distances of the texel centers from the pixel center). When mipmapping is enabled, the nearest mip level is selected.
- *Trilinear*. In this approach, bilinear filtering is used on each of the two nearest mip levels (one higher-res than the ideal and the other lower-res), and these results are then linearly interpolated. This eliminates abrupt visual boundaries between mip levels on-screen.
- *Anisotropic*. Both bilinear and trilinear filtering sample 2×2 square blocks of texels. This is the right thing to do when the textured surface is being viewed head-on, but it's incorrect when the surface is at an oblique angle relative to the virtual screen plane. Anisotropic filtering samples texels within a trapezoidal region corresponding to the view

angle, thereby increasing the quality of textured surfaces when viewed at an angle.

II.1.2.6 Materials

A *material* is a complete description of the visual properties of a mesh. This includes a specification of the textures that are mapped to its surface and also various higher-level properties, such as which shader programs to use when rendering the mesh, the input parameters to those shaders and other parameters that control the functionality of the graphics acceleration hardware itself.

While technically part of the surface properties description, vertex attributes are not considered to be part of the material. However, they come along for the ride with the mesh, so a mesh-material pair contains all the information we need to render the object. Mesh-material pairs are sometimes called *render packets*, and the term “geometric primitive” is sometimes extended to encompass mesh-material pairs as well.

A 3D model typically uses more than one material. For example, a model of a human would have separate materials for the hair, skin, eyes, teeth and various kinds of clothing. For this reason, a mesh is usually divided into *sub-meshes*, each mapped to a single material. The OGRE rendering engine implements this design via its `Ogre::SubMesh` class.



Figure II.19. A variation on the classic “Cornell box” scene illustrating how realistic lighting can make even the simplest scene appear photorealistic.



Figure 11.20. A scene from *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4) rendered without textures. (See Color Plate XV.)

11.1.3 Lighting Basics

Lighting is at the heart of all CG rendering. Without good lighting, an otherwise beautifully modeled scene will look flat and artificial. Likewise, even the simplest of scenes can be made to look extremely realistic when it is lit accurately. The classic “Cornell box” scene, shown in Figure 11.19, is an excellent example of this.

The sequence of screenshots from Naughty Dog’s *The Last of Us: Remastered* is another good illustration of the importance of lighting. In Figure 11.20, the scene is rendered without textures. Figure 11.21 shows the same scene with diffuse textures applied. The fully lit scene is shown in Figure 11.22. Notice the marked jump in realism when lighting is applied to the scene.

The term *shading* is often used as a loose generalization of lighting plus other visual effects. As such, “shading” encompasses procedural deformation of vertices to simulate the motion of a water surface, generation of hair curves or fur shells, tessellation of high-order surfaces, and pretty much any other calculation that’s required to render a scene.

In the following sections, we’ll lay the foundations of lighting that we’ll need in order to understand graphics hardware and the rendering pipeline. We’ll return to the topic of lighting in Section 11.3, where we’ll survey some advanced lighting and shading techniques.

11.1.3.1 Local and Global Illumination Models

Rendering engines use various mathematical models of light-surface and light-volume interactions called *light transport models*. The simplest models only account for *direct lighting* in which light is emitted, bounces off a single



Figure II.21. The same scene from *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4) with only diffuse textures applied. (See Color Plate XVI.)



Figure II.22. Scene from *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4) with full lighting. (See Color Plate XVII.)

object in the scene, and then proceeds directly to the imaging plane of the virtual camera. Such simple models are called *local illumination models*, because only the local effects of light on a single object are considered; objects do not affect one another's appearance in a local lighting model. Not surprisingly, local models were the first to be used in games, and they are still in use today—local lighting can produce surprisingly realistic results in some circumstances.

True photorealism can only be achieved by accounting for *indirect lighting*, where light bounces multiple times off many surfaces before reaching the virtual camera. Lighting models that account for indirect lighting are called *global illumination models*. Some global illumination models are targeted at simulating one specific visual phenomenon, such as producing realistic shadows, modeling reflective surfaces, accounting for interreflection between objects (where the color of one object affects the colors of surrounding objects), and modeling caustic effects (the intense reflections from water or a shiny metal surface). Other global illumination models attempt to provide a holistic account of a wide range of optical phenomena. Ray tracing and radiosity methods are examples of such technologies.

Global illumination is described completely by a mathematical formulation known as the *rendering equation* or *shading equation*. It was introduced in 1986 by J. T. Kajiya as part of a seminal SIGGRAPH paper. In a sense, every rendering technique can be thought of as a full or partial solution to the rendering equation, although they differ in their fundamental approach to solving it and in the assumptions, simplifications and approximations they make. See http://en.wikipedia.org/wiki/Rendering_equation, [10], [2] and virtually any other text on advanced rendering and lighting for more details on the rendering equation.

11.1.3.2 The Phong Lighting Model

The most common local lighting model employed by game rendering engines is the *Phong* reflection model. It models the light reflected from a surface as a sum of three distinct terms:

- The *ambient* term models the overall lighting level of the scene. It is a gross approximation of the amount of indirect bounced light present in the scene. Indirect bounces are what cause regions in shadow not to appear totally black.
- The *diffuse* term accounts for light that is reflected uniformly in all directions from each direct light source. This is a good approximation to the way in which real light bounces off a matte surface, such as a block of wood or a piece of cloth.
- The *specular* term models the bright highlights we sometimes see when viewing a glossy surface. Specular highlights occur when the viewing angle is closely aligned with a path of direct reflection from a light source.

Figure 11.23 shows how the ambient, diffuse and specular terms add together to produce the final intensity and color of a surface.

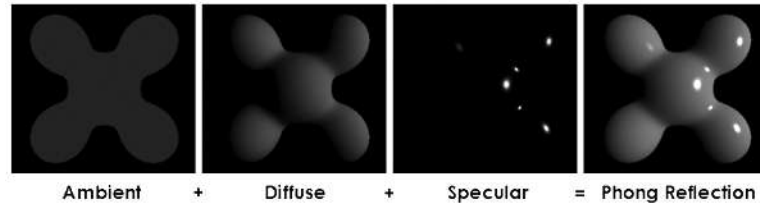


Figure 11.23. Ambient, diffuse and specular terms are summed to calculate Phong reflection.

To calculate Phong reflection at a specific point on a surface, we require a number of input parameters. The Phong model is normally applied to all three color channels (R, G and B) independently, so all of the color parameters in the following discussion are three-element vectors. The inputs to the Phong model are:

- the viewing direction vector $\mathbf{V} = [V_x \ V_y \ V_z]$, which extends from the reflection point to the virtual camera's focal point (i.e., the negation of the camera's world-space "front" vector);
- the ambient light intensity for the three color channels, $\mathbf{A} = [A_R \ A_G \ A_B]$;
- the surface normal $\mathbf{N} = [N_x \ N_y \ N_z]$ at the point the light ray impinges on the surface;
- the surface reflectance properties, which are
 - the ambient reflectivity $\mathbf{k}_A = [k_{AR} \ k_{AG} \ k_{AB}]$,
 - the diffuse reflectivity $\mathbf{k}_D = [k_{DR} \ k_{DG} \ k_{DB}]$,
 - the specular reflectivity $\mathbf{k}_S = [k_{SR} \ k_{SG} \ k_{SB}]$,
 - a specular "glossiness" exponent α ;
- and, for each light source i ,
 - the light's color and intensity $\mathbf{C}_i = [C_{iR} \ C_{iG} \ C_{iB}]$,
 - the direction vector $\mathbf{L}_i$ from the reflection point to the light source.

In the Phong model, the intensity $\mathbf{I}$ of light reflected from a point can be expressed with the following vector equation:

$$\mathbf{I} = (\mathbf{k}_A \otimes \mathbf{A}) + \sum_i [\mathbf{k}_D(\mathbf{N} \cdot \mathbf{L}_i) + \mathbf{k}_S(\mathbf{R}_i \cdot \mathbf{V})^\alpha] \otimes \mathbf{C}_i,$$

where the sum is taken over all lights i affecting the point in question. Recall that the operator $\otimes$ represents the *component-wise* multiplication of two vectors

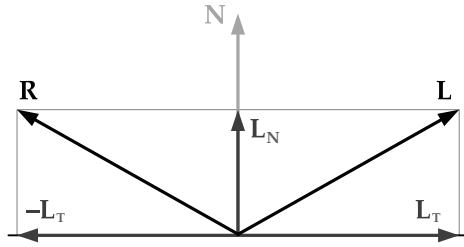


Figure 11.24. Calculation of the reflected lighting vector $\mathbf{R}$ from the original lighting vector $\mathbf{L}$ and the surface normal $\mathbf{N}$.

(the so-called Hadamard product). This expression can be broken into three scalar equations, one for each color channel, as follows:

$$\begin{aligned} I_R &= k_{AR}A_R + \sum_i [k_{DR}(\mathbf{N} \cdot \mathbf{L}_i) + k_{SR}(\mathbf{R}_i \cdot \mathbf{V})^\alpha] C_{iR}, \\ I_G &= k_{AG}A_G + \sum_i [k_{DG}(\mathbf{N} \cdot \mathbf{L}_i) + k_{SG}(\mathbf{R}_i \cdot \mathbf{V})^\alpha] C_{iG}, \\ I_B &= k_{AB}A_B + \sum_i [k_{DB}(\mathbf{N} \cdot \mathbf{L}_i) + k_{SB}(\mathbf{R}_i \cdot \mathbf{V})^\alpha] C_{iB}. \end{aligned}$$

In these equations, the vector $\mathbf{R}_i = [R_{ix} \ R_{iy} \ R_{iz}]$ is the *reflection* of the light ray's direction vector $\mathbf{L}_i$ about the surface normal $\mathbf{N}$.

The vector $\mathbf{R}_i$ can be easily calculated via a bit of vector math (see Figure 11.24). Any vector can be expressed as a sum of its normal and tangential components. For example, we can break up the light direction vector $\mathbf{L}$ as follows:

$$\mathbf{L} = \mathbf{L}_N + \mathbf{L}_T.$$

We know that the dot product $(\mathbf{N} \cdot \mathbf{L})$ represents the projection of $\mathbf{L}$ normal to the surface (a scalar quantity). So the normal component $\mathbf{L}_N$ is just the unit normal vector $\mathbf{N}$ scaled by this dot product:

$$\mathbf{L}_N = (\mathbf{N} \cdot \mathbf{L})\mathbf{N}.$$

The reflected vector $\mathbf{R}$ has the same normal component as $\mathbf{L}$ but the *opposite* tangential component $(-\mathbf{L}_T)$. So we can find $\mathbf{R}$ as follows:

$$\begin{aligned} \mathbf{R} &= \mathbf{L}_N - \mathbf{L}_T \\ &= \mathbf{L}_N - (\mathbf{L} - \mathbf{L}_N) \\ &= 2\mathbf{L}_N - \mathbf{L}; \\ \mathbf{R} &= 2(\mathbf{N} \cdot \mathbf{L})\mathbf{N} - \mathbf{L} \end{aligned}$$

This equation can be used to find all of the $\mathbf{R}_i$ values corresponding to the light directions $\mathbf{L}_i$.

Blinn-Phong

The *Blinn-Phong* lighting model is a variation on Phong shading that calculates specular reflection in a slightly different way. We define the vector $\mathbf{H}$ to be the vector that lies halfway between the view vector $\mathbf{V}$ and the light direction vector $\mathbf{L}$. The Blinn-Phong specular component is then $(\mathbf{N} \cdot \mathbf{H})^a$, as opposed to Phong's $(\mathbf{R} \cdot \mathbf{V})^\alpha$. The exponent a is slightly different than the Phong exponent α , but its value is chosen in order to closely match the equivalent Phong specular term.

The Blinn-Phong model offers increased runtime efficiency at the cost of some accuracy, although it actually matches empirical results more closely than Phong for some kinds of surfaces. The Blinn-Phong model was used almost exclusively in early computer games and was hard-wired into the fixed-function pipelines of early GPUs. See http://en.wikipedia.org/wiki/Blinn%E2%80%93Phong_shading_model for more details.

BRDF Plots

The three terms in the Phong lighting model are special cases of a general local reflection model known as a *bidirectional reflection distribution function* (BRDF). A BRDF calculates the ratio of the outgoing (reflected) radiance along a given viewing direction $\mathbf{V}$ to the incoming irradiance along the incident ray $\mathbf{L}$.

A BRDF can be visualized as a hemispherical plot, where the radial distance from the origin represents the intensity of the light that would be seen if the reflection point were viewed from that direction. The *diffuse* Phong reflection term is $k_D(\mathbf{N} \cdot \mathbf{L})$. This term only accounts for the incoming illumination ray $\mathbf{L}$, not the viewing angle $\mathbf{V}$. Hence the value of this term is the same for all viewing angles. If we were to plot this term as a function of the viewing angle in three dimensions, it would look like a hemisphere centered on the point at which we are calculating the Phong reflection. This is shown in two dimensions in Figure 11.25.

The specular term of the Phong model is $k_D(\mathbf{R} \cdot \mathbf{V})^\alpha$. This term is dependent on both the illumination direction $\mathbf{L}$ and the viewing direction $\mathbf{V}$. It produces a specular “hot spot” when the viewing angle aligns closely with the reflection $\mathbf{R}$ of the illumination direction $\mathbf{L}$ about the surface normal. However, its contribution falls off very quickly as the viewing angle diverges from the reflected illumination direction. This is shown in two dimensions in Figure 11.26.

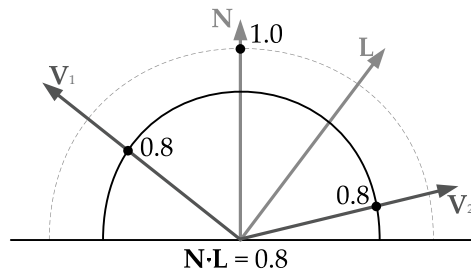


Figure II.25. The diffuse term of the Phong reflection model is dependent upon $\mathbf{N} \cdot \mathbf{L}$ but is independent of the viewing angle $\mathbf{V}$.

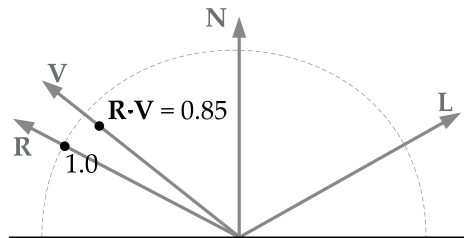


Figure II.26. The specular term of the Phong reflection model is at its maximum when the viewing angle $\mathbf{V}$ coincides with the reflected light direction $\mathbf{R}$ and drops off quickly as $\mathbf{V}$ diverges from $\mathbf{R}$.

II.1.3.3 Modeling Light Sources

In addition to modeling the light's interactions with surfaces, we need to describe the sources of light in the scene. As with all things in real-time rendering, we approximate real-world light sources using various simplified models.

Static Lighting

The fastest lighting calculation is the one you don't do at all. Lighting is therefore performed offline whenever possible. We can precalculate Phong reflection at the vertices of a mesh and store the results as diffuse vertex color attributes. We can also precalculate lighting on a per-pixel basis and store the results in a kind of texture map known as a *light map*. At runtime, the light map texture is projected onto the objects in the scene in order to determine the light's effects on them.

You might wonder why we don't just bake lighting information directly into the diffuse textures in the scene. There are a few reasons for this. For one thing, diffuse texture maps are often tiled and/or repeated throughout a scene,

so baking lighting into them wouldn't be practical. Instead, a single light map is usually generated per light source and applied to any objects that fall within that light's area of influence. This approach permits dynamic objects to move past a light source and be properly illuminated by it. It also means that our light maps can be of a different (often lower) resolution than our diffuse texture maps. Finally, a "pure" light map usually compresses better than one that includes diffuse color information.

Ambient Lights

An *ambient light* corresponds to the ambient term in the Phong lighting model. This term is independent of the viewing angle and has no specific direction. An ambient light is therefore represented by a single color, corresponding to the **A** color term in the Phong equation (which is scaled by the surface's ambient reflectivity k_A at runtime). The intensity and color of ambient light may vary from region to region within the game world.

Directional Lights

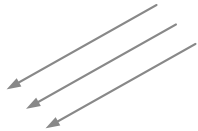


Figure 11.27. Model of a directional light source.

A *directional light* models a light source that is effectively an infinite distance away from the surface being illuminated—like the sun. The rays emanating from a directional light are parallel, and the light itself does not have any particular location in the game world. A directional light is therefore modeled as a light color **C** and a direction vector **L**. A directional light is depicted in Figure 11.27.

Point (Omnidirectional) Lights

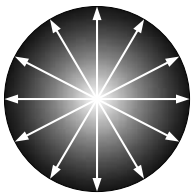


Figure 11.28. Model of a point light source.

A *point light* (*omnidirectional light*) has a distinct position in the game world and radiates uniformly in all directions. The intensity of the light is usually considered to fall off with the square of the distance from the light source, and beyond a predefined maximum radius its effects are simply clamped to zero. A point light is modeled as a light position **P**, a source color/intensity **C** and a maximum radius $r_{\max}$. The rendering engine only applies the effects of a point light to those surfaces that fall within its sphere of influence (a significant optimization). Figure 11.28 illustrates a point light.

Spot Lights

A *spot light* acts like a point light whose rays are restricted to a cone-shaped region, like a flashlight. Usually two cones are specified with an inner and an outer angle. Within the inner cone, the light is considered to be at full intensity.

The light intensity falls off as the angle increases from the inner to the outer angle, and beyond the outer cone it is considered to be zero. Within both cones, the light intensity also falls off with radial distance. A spot light is modeled as a position $\mathbf{P}$, a source color $\mathbf{C}$, a central direction vector $\mathbf{L}$, a maximum radius $r_{\max}$ and inner and outer cone angles $\theta_{\min}$ and $\theta_{\max}$. Figure 11.29 illustrates a spot light source.

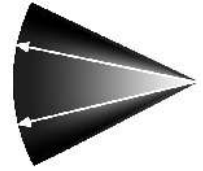


Figure 11.29. Model of a spot light source.

Area Lights

All of the light sources we've discussed thus far radiate from an idealized point, either at infinity or locally. A real light source almost always has a nonzero area—this is what gives rise to the umbra and penumbra in the shadows it casts.

Rather than trying to model area lights explicitly, CG engineers often use various “tricks” to account for their behavior. For example to simulate a penumbra, we might cast multiple shadows and blend the results, or we might blur the edges of a sharp shadow in some manner.

Emissive Objects

Some surfaces in a scene are themselves light sources. Examples include flashlights, glowing crystal balls, flames from a rocket engine and so on. Glowing surfaces can be modeled using an *emissive texture map*—a texture whose colors are always at full intensity, independent of the surrounding lighting environment. Such a texture could be used to define a neon sign, a car's headlights and so on.

Some kinds of emissive objects are rendered by combining multiple techniques. For example, a flashlight might be rendered using an emissive texture for when you're looking head-on into the beam, a colocated spot light that casts light into the scene, a yellow translucent mesh to simulate the light cone, some camera-facing transparent cards to simulate lens flare (or a *bloom* effect if high dynamic range lighting is supported by the engine), and a projected texture to produce the caustic effect that a flashlight has on the surfaces it illuminates. The flashlight in *Luigi's Mansion* is a great example of this kind of effect combination, as shown in Figure 11.30.

11.1.4 The Virtual Camera

In computer graphics, the virtual camera is much simpler than a real camera or the human eye. We treat the camera as an ideal focal point with a rectangular virtual sensing surface called the *imaging rectangle* floating some small distance



Figure II.30. The flashlight in *Luigi's Mansion* by Nintendo (Wii) is composed of numerous visual effects, including a cone of translucent geometry for the beam, a dynamic spot light to cast light into the scene, an emissive texture on the lens and camera-facing cards for the lens flare. (See Color Plate XVIII.)

in front of it. The imaging rectangle consists of a grid of square or rectangular virtual light sensors, each corresponding to a single pixel on-screen. Rendering can be thought of as the process of determining what color and intensity of light would be recorded by each of these virtual sensors.

II.I.4.1 View Space

The focal point of the virtual camera is the origin of a 3D coordinate system known as *view space* or *camera space*. The camera usually “looks” down the positive or negative z -axis in view space, with y up and x to the left or right. Typical left- and right-handed view-space axes are illustrated in Figure 11.31.

The camera’s position and orientation can be specified using a view-to-world matrix, just as a mesh instance is located in the scene with its model-to-world matrix. If we know the position vector and three unit basis vectors of camera space, expressed in world-space coordinates, the view-to-world matrix can be written as follows, in a manner analogous to that used to construct a model-to-world matrix:

$$\mathbf{M}_{V \rightarrow W} = \left[\begin{array}{c|c} \mathbf{i}_V & 0 \\ \mathbf{j}_V & 0 \\ \mathbf{k}_V & 0 \\ \hline \mathbf{t}_V & 1 \end{array} \right].$$

When rendering a triangle mesh, its vertices are transformed first from model space to world space, and then from world space to view space. To perform this latter transformation, we need the world-to-view matrix, which is the inverse of the view-to-world matrix. This matrix is sometimes called the *view matrix*:

$$\mathbf{M}_{W \rightarrow V} = \mathbf{M}_{V \rightarrow W}^{-1} = \mathbf{M}_{\text{view}}.$$

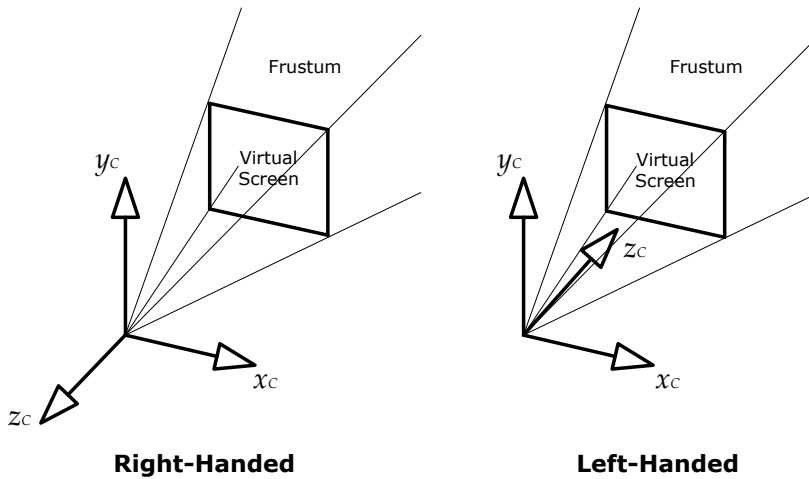


Figure 11.31. Left- and right-handed camera-space axes.

Be careful here. The fact that the camera's matrix is inverted relative to the matrices of the objects in the scene is a common point of confusion and bugs among new game developers.

The world-to-view matrix is often concatenated to the model-to-world matrix prior to rendering a particular mesh instance. This combined matrix is called the *model-view matrix* in OpenGL. We precalculate this matrix so that the rendering engine only needs to do a single matrix multiply when transforming vertices from model space into view space:

$$\mathbf{M}_{M \rightarrow V} = \mathbf{M}_{M \rightarrow W} \mathbf{M}_{W \rightarrow V} = \mathbf{M}_{\text{modelview}}.$$

11.1.4.2 Projections

In order to render a 3D scene onto a 2D image plane, we use a special kind of transformation known as a *projection*. The *perspective* projection is the most common projection in computer graphics, because it mimics the kinds of images produced by a typical camera. With this projection, objects appear smaller the farther away they are from the camera—an effect known as *perspective foreshortening*.

The length-preserving *orthographic* projection is also used by some games, primarily for rendering *plan views* (e.g., front, side and top) of 3D models or game levels for editing purposes, and for overlaying 2D graphics onto the screen for heads-up displays and the like. Figure 11.32 illustrates how a cube would look when rendered with these two types of projections.

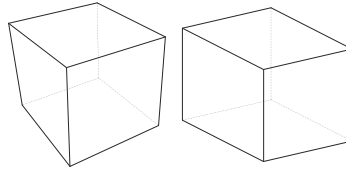


Figure 11.32. A cube rendered using a perspective projection (on the left) and an orthographic projection (on the right).

11.1.4.3 The View Volume and the Frustum

The region of space that the camera can “see” is known as the view volume. A view volume is defined by six planes. The *near plane* corresponds to the virtual image-sensing surface. The four side planes correspond to the edges of the virtual screen. The *far plane* is used as a rendering optimization to ensure that extremely distant objects are not drawn. It also provides an upper limit for the depths that will be stored in the depth buffer (see Section 11.1.4.8).

When rendering the scene with a *perspective* projection, the shape of the view volume is a truncated pyramid known as a *frustum*. When using an orthographic projection, the view volume is a rectangular prism. Perspective and orthographic view volumes are illustrated in Figure 11.33 and Figure 11.34, respectively.

The six planes of the view volume can be represented compactly using six four-element vectors $(n_{ix}, n_{iy}, n_{iz}, d_i)$, where $\mathbf{n} = (n_x, n_y, n_z)$ is the plane normal and d is its perpendicular distance from the origin. If we prefer the

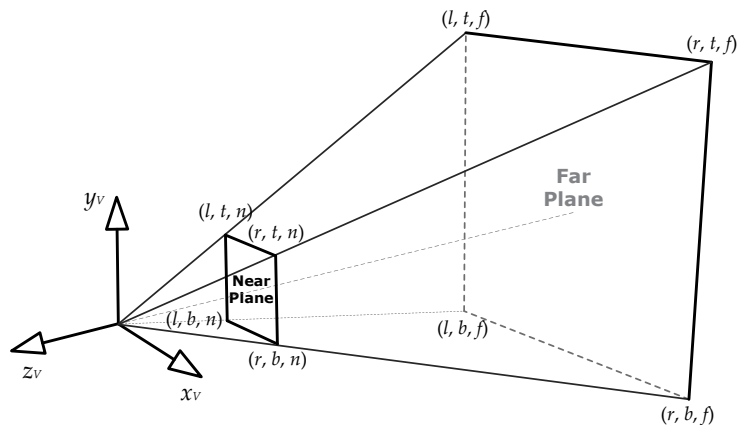


Figure 11.33. A perspective view volume (frustum).

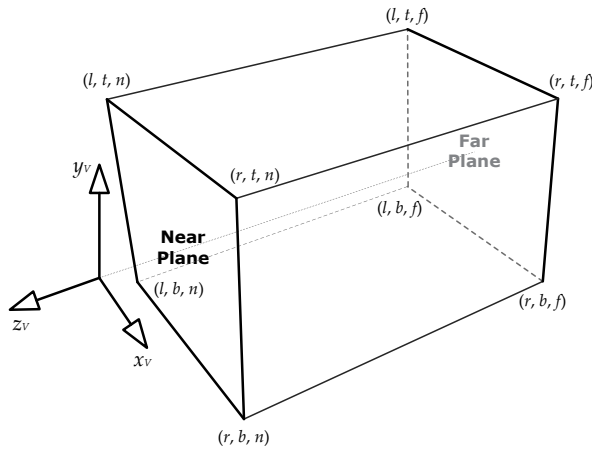


Figure 11.34. An orthographic view volume.

point-normal plane representation, we can also describe the planes with six pairs of vectors $(\mathbf{Q}_i, \mathbf{n}_i)$, where $\mathbf{Q}$ is the arbitrary point on the plane and $\mathbf{n}$ is the plane normal. (In both cases, i is an index representing the six planes.)

11.1.4.4 Projection and Homogeneous Clip Space

Both perspective and orthographic projections transform points in view space into a coordinate space called *homogeneous clip space*. This three-dimensional space is really just a warped version of view space. The purpose of clip space is to convert the camera-space view volume into a canonical view volume that is independent both of the kind of *projection* used to convert the 3D scene into 2D screen space, and of the *resolution* and *aspect ratio* of the screen onto which the scene is going to be rendered.

In clip space, the canonical view volume is a rectangular prism extending from -1 to $+1$ along the x - and y -axes. Along the z -axis, the view volume extends either from -1 to $+1$ (OpenGL) or from 0 to 1 (DirectX). We call this coordinate system “clip space” because the view volume planes are axis-aligned, making it convenient to *clip* triangles to the view volume in this space (even when a perspective projection is being used). The canonical clip-space view volume for OpenGL is depicted in Figure 11.35. Notice that the z -axis of clip space goes into the screen, with y up and x to the right. In other words, homogeneous clip space is usually *left-handed*. A left-handed convention is used here because it causes increasing z values to correspond to increasing *depth* into the screen, with y increasing up and x increasing to the right as usual.

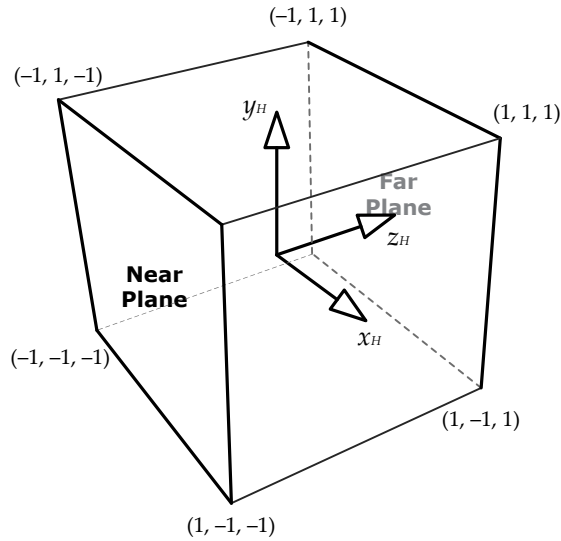


Figure II.35. The canonical view volume in homogeneous clip space.

Perspective Projection

An excellent explanation of perspective projection is given in Section 4.5.1 of [32], so we won't repeat it here. Instead, we'll simply present the perspective projection matrix $\mathbf{M}_{V \rightarrow H}$ below. (The subscript $V \rightarrow H$ indicates that this matrix transforms vertices from view space into homogeneous clip space.) If we take view space to be right-handed, then the near plane intersects the z -axis at $z = -n$, and the far plane intersects it at $z = -f$. The virtual screen's left, right, bottom, and top edges lie at $x = l$, $x = r$, $y = b$ and $y = t$ on the near plane, respectively. (Typically the virtual screen is centered on the camera-space z -axis, in which case $l = -r$ and $b = -t$, but this isn't always the case.) Using these definitions, the perspective projection matrix for OpenGL is as follows:

$$\mathbf{M}_{V \rightarrow H} = \begin{bmatrix} \left(\frac{2n}{r-l} \right) & 0 & 0 & 0 \\ 0 & \left(\frac{2n}{t-b} \right) & 0 & 0 \\ \left(\frac{r+l}{r-l} \right) & \left(\frac{t+b}{t-b} \right) & \left(-\frac{f+n}{f-n} \right) & -1 \\ 0 & 0 & \left(-\frac{2nf}{f-n} \right) & 0 \end{bmatrix}.$$

DirectX defines the z -axis extents of the clip-space view volume to lie in

the range $[0, 1]$ rather than in the range $[-1, 1]$ as OpenGL does. We can easily adjust the perspective projection matrix to account for DirectX's conventions as follows:

$$(\mathbf{M}_{V \rightarrow H})_{\text{DirectX}} = \begin{bmatrix} \left(\frac{2n}{r-l}\right) & 0 & 0 & 0 \\ 0 & \left(\frac{2n}{t-b}\right) & 0 & 0 \\ \left(\frac{r+l}{r-l}\right) & \left(\frac{t+b}{t-b}\right) & \left(-\frac{f}{f-n}\right) & -1 \\ 0 & 0 & \left(-\frac{nf}{f-n}\right) & 0 \end{bmatrix}.$$

Division by z

Perspective projection results in each vertex's x - and y -coordinates being divided by its z -coordinate. This is what produces perspective foreshortening. To understand why this happens, consider multiplying a view-space point $\mathbf{p}_V$ expressed in four-element homogeneous coordinates by the OpenGL perspective projection matrix:

$$\begin{aligned} \mathbf{p}_H &= \mathbf{p}_V \mathbf{M}_{V \rightarrow H} \\ &= \begin{bmatrix} p_{Vx} & p_{Vy} & p_{Vz} & 1 \end{bmatrix} \begin{bmatrix} \left(\frac{2n}{r-l}\right) & 0 & 0 & 0 \\ 0 & \left(\frac{2n}{t-b}\right) & 0 & 0 \\ \left(\frac{r+l}{r-l}\right) & \left(\frac{t+b}{t-b}\right) & \left(-\frac{f+n}{f-n}\right) & -1 \\ 0 & 0 & \left(-\frac{2nf}{f-n}\right) & 0 \end{bmatrix}. \end{aligned}$$

The result of this multiplication takes the form

$$\mathbf{p}_H = \begin{bmatrix} a & b & c & -p_{Vz} \end{bmatrix}. \quad (11.1)$$

When we convert any homogeneous vector into three-dimensional coordinates, the x -, y - and z -components are divided by the w -component:

$$\begin{bmatrix} x & y & z & w \end{bmatrix} \equiv \begin{bmatrix} \frac{x}{w} & \frac{y}{w} & \frac{z}{w} \end{bmatrix}.$$

So, after dividing Equation (11.1) by the homogeneous w -component, which is really just the negative view-space z -coordinate $-p_{Vz}$, we have:

$$\begin{aligned} \mathbf{p}_H &= \begin{bmatrix} \frac{a}{-p_{Vz}} & \frac{b}{-p_{Vz}} & \frac{c}{-p_{Vz}} \end{bmatrix} \\ &= \begin{bmatrix} p_{Hx} & p_{Hy} & p_{Hz} \end{bmatrix}. \end{aligned}$$

Thus, the homogeneous clip-space coordinates have been divided by the view-space z -coordinate, which is what causes perspective foreshortening.

Perspective-Correct Vertex Attribute Interpolation

In Section 11.1.2.4, we learned that vertex attributes are interpolated in order to determine appropriate values for them within the interior of a triangle. Attribute interpolation is performed in *screen space*. We iterate over each pixel of the screen and attempt to determine the value of each attribute at the corresponding location *on the surface of the triangle*. When rendering a scene with a perspective projection, we must do this very carefully so as to account for perspective foreshortening. This is known as *perspective-correct* attribute interpolation.

A derivation of perspective-correct interpolation is beyond our scope, but suffice it to say that we must divide our interpolated attribute values by the corresponding z -coordinates (depths) at each vertex. For any pair of vertex attributes A_1 and A_2 , we can write the interpolated attribute at a percentage t of the distance between them as follows:

$$\frac{A}{p_z} = (1 - t) \left(\frac{A_1}{p_{1z}} \right) + t \left(\frac{A_2}{p_{2z}} \right) = \text{LERP} \left(\frac{A_1}{p_{1z}}, \frac{A_2}{p_{2z}}, t \right).$$

Refer to [32] for an excellent derivation of the math behind perspective-correct attribute interpolation.

Orthographic Projection

An orthographic projection is performed by the following matrix:

$$(\mathbf{M}_{V \rightarrow H})_{\text{ortho}} = \begin{bmatrix} \left(\frac{2}{r-l} \right) & 0 & 0 & 0 \\ 0 & \left(\frac{2}{t-b} \right) & 0 & 0 \\ 0 & 0 & \left(-\frac{2}{f-n} \right) & 0 \\ \left(-\frac{r+l}{r-l} \right) & \left(-\frac{t+b}{t-b} \right) & \left(-\frac{f+n}{f-n} \right) & 1 \end{bmatrix}.$$

This is just an everyday scale-and-translate matrix. (The upper-left 3×3 contains a diagonal nonuniform scaling matrix, and the lower row contains the translation.) Since the view volume is a rectangular prism in both view space and clip space, we need only scale and translate our vertices to convert from one space to the other.

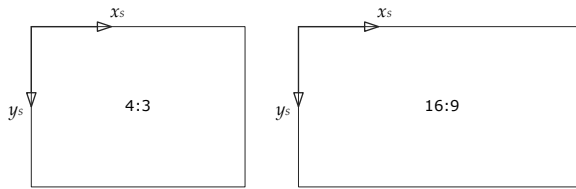


Figure 11.36. The two most prevalent screen-space aspect ratios are 4:3 and 16:9.

11.1.4.5 Screen Space and Aspect Ratios

Screen space is a two-dimensional coordinate system whose axes are measured in terms of screen pixels. The x -axis typically points to the right, with the origin at the top-left corner of the screen and y pointing down. (The reason for the inverted y -axis is that CRT monitors scan the screen from top to bottom.) The ratio of screen width to screen height is known as the *aspect ratio*. The most common aspect ratios are 4:3 (the aspect ratio of a traditional television screen) and 16:9 (the aspect ratio of a movie screen or HDTV). These aspect ratios are illustrated in Figure 11.36.

We can render triangles expressed in homogeneous clip space by simply drawing their (x, y) coordinates and ignoring z . But before we do, we scale and shift the clip-space coordinates so that they lie in screen space rather than within the normalized unit square. This scale-and-shift operation is known as *screen mapping*.

11.1.4.6 The Frame Buffer

The final rendered image is stored in a bitmapped color buffer known as the *frame buffer*. Pixel colors are usually stored in RGBA8888 format, although other frame buffer formats are supported by most graphics cards as well. Some common formats include RGB565, RGB5551, and one or more paletted modes.

The display hardware (CRT, flat-screen monitor, HDTV, etc.) reads the contents of the frame buffer at a periodic rate of 60 Hz for NTSC televisions used in North America and Japan, or 50 Hz for PAL/SECAM televisions used in Europe and many other places in the world. Rendering engines typically maintain at least two frame buffers. While one is being scanned by the display hardware, the other one can be updated by the rendering engine. This is known as *double buffering*. By swapping or “flipping” the two buffers during the *vertical blanking interval* (the period during which the CRT’s electron gun is being reset to the top-left corner of the screen), double buffering ensures that the display hardware always scans the complete frame buffer. This avoids a jarring effect

known as *tearing*, in which the upper portion of the screen displays the newly rendered image while the bottom shows the remnants of the previous frame's image.

Some engines make use of three frame buffers—a technique aptly known as *triple buffering*. This is done so that the rendering engine can start work on the next frame, even while the previous frame is still being scanned by the display hardware. For example, the hardware might still be scanning buffer A when the engine finishes drawing buffer B. With triple buffering, it can proceed to render a new frame into buffer C, rather than idling while it waits for the display hardware to finish scanning buffer A.

Render Targets

Any buffer into which the rendering engine draws graphics is known as a *render target*. As we'll see later in this chapter, rendering engines make use of all sorts of other off-screen render targets in addition to the frame buffers. These include the depth buffer, the stencil buffer and various other buffers used for storing intermediate rendering results.

II.1.4.7 Triangle Rasterization and Fragments

To produce an image of a triangle on-screen, we need to fill in the pixels it overlaps. This process is known as rasterization. During rasterization, the triangle's surface is broken into pieces called *fragments*, each one representing a small region of the triangle's surface that corresponds to a single pixel on the screen. (In the case of multisample antialiasing, a fragment corresponds to a *portion* of a pixel—see below.)

A fragment is like a pixel in training. Before it is written into the frame buffer, it must pass a number of tests (described in more depth below). If it fails any of these tests, it will be discarded. Fragments that pass the tests are shaded (i.e., their colors are determined), and the fragment color is either written into the frame buffer or blended with the pixel color that's already there. Figure 11.37 illustrates how a fragment becomes a pixel.

II.1.4.8 Occlusion and the Depth Buffer

When rendering two triangles that overlap each other in screen space, we need some way of ensuring that the triangle that is closer to the camera will appear on top. We could accomplish this by always rendering our triangles in back-to-front order (the so-called painter's algorithm). However, as shown in Figure 11.38, this doesn't work if the triangles are intersecting one another.

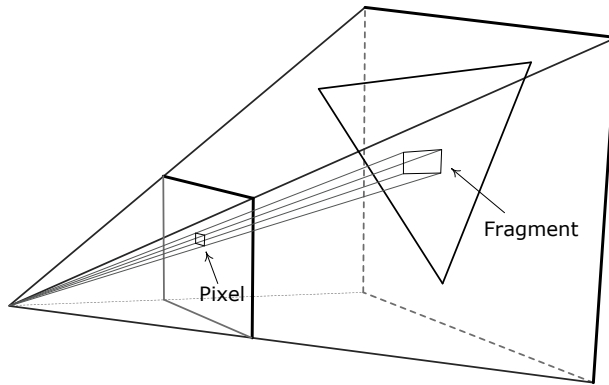


Figure II.37. A fragment is a small region of a triangle corresponding to a pixel on the screen. It passes through the rendering pipeline and is either discarded or its color is written into the frame buffer.

To implement triangle occlusion properly, independent of the order in which the triangles are rendered, rendering engines use a technique known as *depth buffering* or *z-buffering*. The depth buffer is a full-screen buffer that typically contains 24-bit integer or (more rarely) floating-point depth information for each pixel in the frame buffer. (The depth buffer is usually stored in a 32-bits-per-pixel format, with a 24-bit depth value and an 8-bit stencil value packed into each pixel's 32-bit quadword.) Every fragment has a z-coordinate that measures its depth "into" the screen. (The depth of a fragment is found by interpolating the depths of the triangle's vertices.) When a fragment's color is written into the frame buffer, its depth is stored into the corresponding pixel of the depth buffer. When another fragment (from another triangle) is drawn into the same pixel, the engine compares the new fragment's depth to the depth already present in the depth buffer. If the fragment is closer to the camera (i.e.,

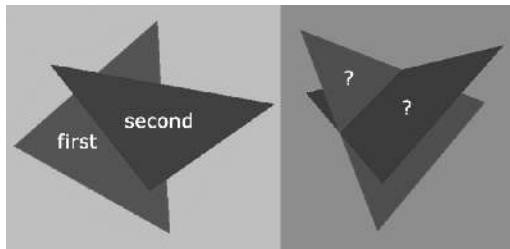


Figure II.38. The painter's algorithm renders triangles in a back-to-front order to produce proper triangle occlusion. However, the algorithm breaks down when triangles intersect one another.

if it has a smaller depth), it overwrites the pixel in the frame buffer. Otherwise the fragment is discarded.

z-Fighting and the w-Buffer

When rendering parallel surfaces that are very close to one another, it's important that the rendering engine can distinguish between the depths of the two planes. If our depth buffer had infinite precision, this would never be a problem. Unfortunately, a real depth buffer only has limited precision, so the depth values of two planes can collapse into a single discrete value when the planes are close enough together. When this happens, the more-distant plane's pixels start to "poke through" the nearer plane, resulting in a noisy effect known as *z-fighting*.

To reduce *z-fighting* to a minimum across the entire scene, we would like to have equal precision whether we're rendering surfaces that are close to the camera or far away. However, with *z-buffering* this is not the case. The precision of clip-space *z*-depths (p_{H_z}) are not evenly distributed across the entire range from the near plane to the far plane, because of the division by the view-space *z*-coordinate. Because of the shape of the $1/z$ curve, most of the depth buffer's precision is concentrated near the camera.

The plot of the function $p_{H_z} = 1/p_{V_z}$ shown in Figure 11.39 demonstrates this effect. Near the camera, the distance between two planes in view space Δp_{V_z} gets transformed into a reasonably large delta in clip space Δp_{H_z} . But far from the camera, this same separation gets transformed into a tiny delta in clip space. The result is *z-fighting*, and it becomes rapidly more prevalent as objects get farther away from the camera.

To circumvent this problem, we would like to store *view-space* *z*-coordinates (p_{V_z}) in the depth buffer instead of clip-space *z*-coordinates (p_{H_z}). View-space *z*-coordinates vary linearly with the distance from the camera, so using them as our depth measure achieves uniform precision across the entire depth range.

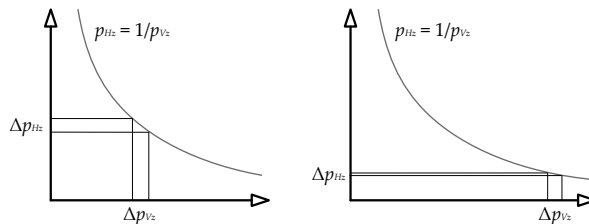


Figure 11.39. A plot of the function $1/p_{V_z}$, showing how most of the precision lies close to the camera.